

1 Introduction

For this assignment the *dry bean* dataset was chosen. It is available on UC Irvine. The dataset contains seventeen features including the target. Sixteen are numeric features extracted by dry beans images captured with a RGB camera.

2 Dataset exploration

After importing the dataset, an exploratory data analysis (EDA) was performed. During this phase, the data types of the sixteen features in the dataset were checked. It was confirmed that all features are numeric; therefore, encoding was not required. Subsequently, the dataset was checked for missing or NaN values, and none were found.

Following this process, the distribution of the target values was studied. As can be seen in Figure 1, the dataset is not uniformly distributed. The Cali, Dermason, and Horoz classes are almost balanced, but the Barbunia and Bombay classes represent the majority of the instances. Lastly, the Seker and Sira classes represent the minority of instances, especially the latter.

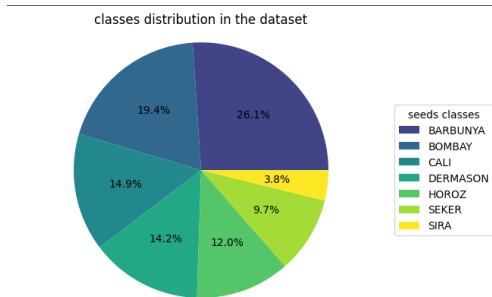


Figure 1: class distribution over in the dataset

3 Dataset splitting and normalization

After the simple EDA process described above an encoding operation is required for the target variable. A label encoder is used to encode string class names into integer. After this the dataset was split into *train*, *validation*, and *test*

sets. This process was performed by subdividing all instances in the dataset into three different groups according to specific percentages. The first split done was *train-test* split, 80% of the dataset is used to train the model. The second split done was *train-validation* split, 20% of instances in the training set are use to validate the model. A seed is used to make this splitting replicable.

After the splitting process there are two required operations. The first is the one hot encoding of the target variable, this is needed to transform the numerical target variable into a matrix representation compatible with the output produced by the neural network. Here in Equation 1 there is an example.

$$\begin{bmatrix} y \\ 5 \\ 2 \\ \vdots \\ 1 \end{bmatrix} \quad \begin{bmatrix} & y \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (1)$$

The second operation done is the normalization of the instances. Given the feature f every instances i of f is normalized using the Equation 2.

$$normalized_f = \frac{i}{max(f)} \quad \forall i \in f \quad (2)$$

This process was applied to all features in the dataset. All sets (training, validation and test) were standardized using the $max(f)$ value extracted from the training set. This procedure is essential to prevent data leakage and avoid bias in the model.

4 Model creation and selection

All the models used in this phase are trained, validated and tested with the normalized and encoded dataset created before. The neural network was created using 4 layers, their properties are described in Table 1: The architecture described in Table 1 was chosen based on the consideration that the dataset consists of 16 features. A single hidden layer was avoided because the relationships among the 16 input features and the

layers	shape	activation function
input layer	16	/
hidden layer 1	64	relu
hidden layer 2	16	relu
output layer	7	softmax

Table 1: Neural network's architecture

output classes are likely complex and non linear. A single layer might lack the capacity to capture these patterns, which could lead to underfitting. Using more than two layers would create a significantly more complex model. This introduces a higher risk of overfitting, where the model would start to memorize the training data instead of learning the general patterns. Therefore, two layers were chosen as a balance between model capacity and generalization.

It was hypothesized that the best performance could be achieved by widening the network, moving from the 16 neurons of the input layer to the 64 neurons of the first hidden layer, followed by a subsequent narrowing. This approach aims to promote better generalization while attempting to limit or delete the phenomenon of overfitting. Consequently, this configuration was preferred over an architecture that progressively narrows from the 16 neurons of the input layer toward the output dimension.

The activation function for the hidden layers is the Rectified Linear Unit, *relu*. This function was chosen over sigmoid, tanh and softmax for two reasons. First, it solves the vanishing gradient problem that afflicts the aforementioned functions. In fact, its derivative is a constant 1 for positive inputs, allowing error signals to propagate back through many layers and enabling learning. Second, it induces a sparse representation by setting all negative inputs to zero. This means that only a subset of neurons is activated, which leads to faster learning and model simplification.

The last layer uses a *softmax* as activation function because we are doing a multinomial classification problem over 7 classes. This function generates a probability distribution over a discrete variable with N possible values. The class

with the highest probability is interpreted as the model prediction.

The selected loss function is *categorical focal cross-entropy*, as it is particularly effective for multinomial classification problems involving imbalanced datasets, such as the dataset used in this report.

The first model was built using the architecture shown in Table 1, with categorical focal crossentropy as the loss function, as mentioned previously. It uses the *Adam* optimizer. After this settings the first model was trained using *epochs*= 10 and *batch size*= 32.

In Figure 2 there are the results obtained by the model. It should be noted that the results shown were obtained after training a new model with the described parameters on a new training set created by concatenating the training and validation sets. Subsequently, the model was tested on the test set. This process was applied for every result analysis performed. The model

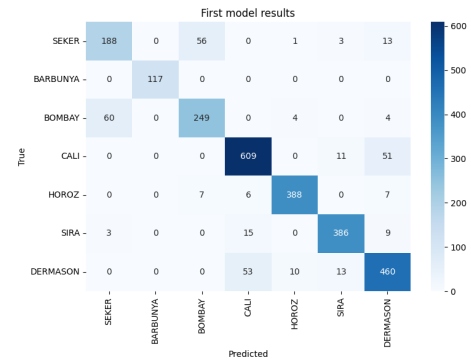


Figure 2: First model results.

exhibits classification difficulties with the class pairs: [seker-bombay] and [cali-dermason]. This may be because the standard deviation of most features related to these classes is very low; consequently, the network struggles to separate and classify them correctly. Furthermore, these particular bean types likely have very similar shapes and dimensions, making them inherently difficult to distinguish. Unexpectedly, no severe classification problems were detected for the minority class, *sira*, which could be due to a higher variance in the data related to that class.

5 Model improvements

The model was then improved by varying several hyperparameters. First, the *batch size* was modified, since the dataset was normalized, a greater amount of noise may have been generated, consequently, it is better to take more examples per batch. The *batch size* was therefore set to 128. At the same time, the number of epochs was increased to 50 because, with a larger *batch size*, the model will require more epochs to converge and minimize the gradient. Figure 3 shows the behaviour of the model's loss function. Observ-

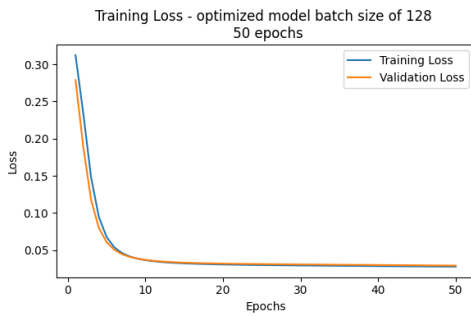


Figure 3: Optimization of the model described before. Batch size= 128 and epochs= 50.

ing Figure 3, it is clear that the number of epochs can be reduced to 20. Furthermore, the learning rate should be slightly increased. This will result in a model that learns in a correct way (Figure 4).

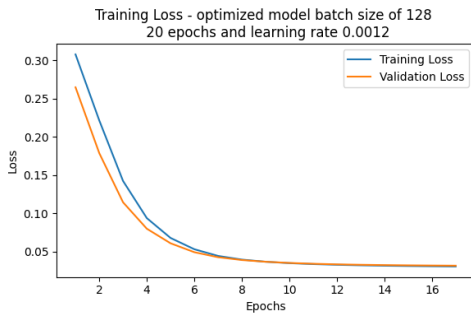


Figure 4: Optimization of the model described before. Batch size= 128 and epochs= 20 and learning rate= 0.0012.

The new model was tested on the same training set that was generated for the unoptimized model, as previously described. The results shown in Figure 5 did not yield significant im-

provements. In fact, comparing its confusion matrix with that of the first model (Figure 2) reveals a mixed outcome, with both improvements and regressions.

Specifically, the class pairs [*seker*, *bombay*] and [*cali*, *dermason*] are still poorly distinguished. Notably, the number of *seker* instances misclassified as *bombay* has increased significantly, while the misclassification of *bombay* as *seker* has decreased. Furthermore, there is an increase in *sira* instances being incorrectly classified as *cali*.

These two described issues might be a result of the model overfitting to the aforementioned classes. This phenomenon may also be a consequence of increasing the batch size.

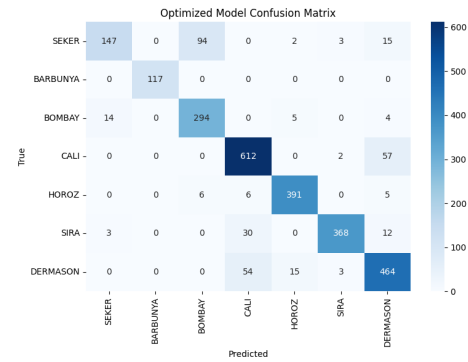


Figure 5: Optimized model results.

Figures 6 and 7 show the accuracy curves for the optimized and unoptimized models. Both confirm what was described previously: there is no significant difference in the models' accuracy. It is important to note that *categorical accuracy* was employed as the metric because it calculates how often predictions match the one-hot labels.

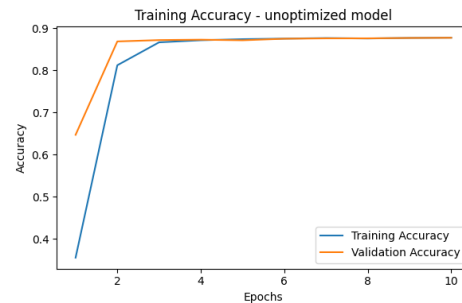


Figure 6: Unoptimized model's accuracy.

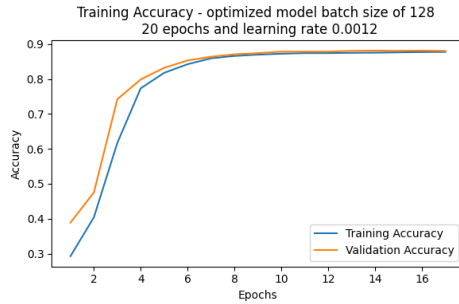


Figure 7: Optimized model's accuracy.

The performance comparison between the two models is also confirmed by the ROC curve in Figure 8, which clearly shows that the models exhibit an identical and comparable behaviour ($AUC = 0.99$) as previously observed in the analysis of the confusion matrices and the accuracy plots. Since the dataset exhibited class imbal-

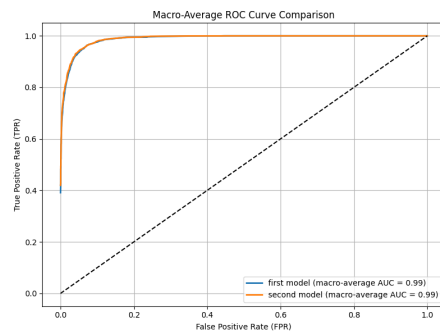


Figure 8: Models' ROC curve.

ance, the ROC curve was computed using the *macro-average*, as it is obtained by averaging the ROC curves across all classes. The curves do not start from the point $(0,0)$ due to interpolation issues that occurred during the macro-averaging of the individual class curves.

6 Future improvements

In the future, the model could be improved by implementing outlier removal during the EDA process and applying PCA (Principal Component Analysis). This would simplify the model and remove highly correlated features.

Regarding the neural network, future work could explore different techniques such as regularization, dynamic learning rate scheduling and

early stopping. Furthermore, it might be useful to change the network's architecture, perhaps by keeping the number of neurons in one hidden layer constant in the next. This process could improve the network's information retention.